Traceability Matrix and Coverage Guide

Document ID: GKB-TRACE-005
Version: 1.0
Scope: General Knowledge Base
Intended use: reusable guidance for traceability, coverage, and evidence review
Owner: Verification and Validation Office

1. Purpose

This guide explains how to build and review traceability between source documents, requirements, design elements, tests, defects, and release evidence.

It is intended for systems engineers, QA engineers, reviewers, and AI-assisted test planning tools.

2. Why Traceability Matters

Traceability answers questions such as:

- Which source document created this requirement?
- Which tests cover this requirement?
- Which requirements have no tests?
- Which tests are not linked to any requirement?
- Which defects affect a release-critical requirement?
- Which evidence supports a release decision?

Traceability is not only a compliance activity. It helps teams find gaps, avoid duplicated work, and explain why testing is sufficient.

3. Common Traceability Objects

Object	Description
Source document	Requirement specification, architecture document, standard, contract, incident log
Section	A heading-level part of a source document
Chunk	Retrieval-sized text segment used by search or AI
Requirement	Atomic testable statement
Test case	Procedure or scenario that verifies one or more requirements
Test result	Execution outcome and evidence
Defect	Failure, nonconformance, or issue
Release decision	Approval, rejection, waiver, or risk acceptance

4. Traceability Relationship Types

Relationship	Meaning
derives from	Requirement derives from source text
refines	Lower-level requirement refines higher-level requirement
satisfies	Design or implementation satisfies a requirement
verifies	Test verifies a requirement
covers	Test case covers a requirement
produces evidence	Test execution produces result evidence
blocks	Defect blocks requirement acceptance
waives	Risk decision accepts missing or partial coverage

5. Requirements Traceability Matrix

A requirements traceability matrix records links between requirements and verification evidence.

Minimum useful columns:

- requirement ID;
- requirement title;
- source document;
- requirement type;
- priority;
- verification method;
- test case ID;
- test result;
- coverage status;
- open defect ID;
- reviewer comment.

Example:

Requirement ID	Source	Verification method	Test case	Result	Coverage
REQ-001	PRD section 4.1	Test	TC-001	Pass	Covered
REQ-002	Security section 3.2	Test	TC-SEC-003	Fail	Covered with defect
REQ-003	Architecture section 5	Analysis	none	Not run	Gap

6. Verification Methods

Typical verification methods:

- inspection;
- analysis;
- demonstration;
- test.

6.1 Inspection

Use inspection when evidence can be obtained by reviewing documents, configuration, code, or logs.

Example:

- verify that an audit log field is present in a database schema.

6.2 Analysis

Use analysis when evidence comes from calculation, model, review, or reasoning.

Example:

- verify that storage capacity is sufficient for 24 hours of buffering.

6.3 Demonstration

Use demonstration when a capability can be shown without precise measurement.

Example:

- demonstrate that an administrator can export a diagnostic bundle.

6.4 Test

Use test when evidence requires controlled execution and observable result.

Example:

- simulate cloud outage and verify telemetry is buffered.

7. Coverage Status Definitions

Status	Meaning
Covered	At least one relevant test or verification item exists
Partially covered	Test exists but does not address all conditions
Gap	No verification item exists
Blocked	Verification cannot proceed due to dependency
Waived	Gap accepted by authorized decision
Not applicable	Requirement not in current release scope

8. Coverage Review Checklist

Review the coverage matrix for:

- requirements with no tests;
- tests with no requirement link;
- high-priority requirements with only weak tests;
- security or safety requirements lacking negative tests;
- performance requirements lacking measurable thresholds;
- duplicate tests covering the same low-risk requirement;
- requirements linked only to manual inspection when execution is needed;
- obsolete tests linked to changed requirements.

9. Handling Coverage Gaps

When a gap is found:

1. Confirm the requirement is in scope.
2. Confirm no existing test already covers it.
3. Decide whether a new test, analysis, inspection, or waiver is appropriate.
4. Create or update the verification item.
5. Link it to the requirement.
6. Recompute coverage.
7. Record reviewer decision.

10. Traceability in AI-Generated Plans

AI-generated test plans must preserve traceability.

The AI should:

- cite source document chunks for requirements;
- link every test case to requirement IDs;
- avoid generating tests without a clear source or risk rationale;
- identify uncovered requirements;
- indicate weak or partial coverage;
- distinguish project-specific requirements from general knowledge guidance.

The AI should not:

- invent source citations;

- mark a requirement covered by a generic test that does not verify it;
- ignore failed or blocked test results;
- mix general standards guidance with project-specific requirements without labeling the difference.

11. Traceability Evidence Package

For a release review, provide:

- requirement baseline;
- test plan;
- coverage matrix;
- test execution report;
- open defect list;
- waivers or risk acceptances;
- reviewer sign-off.

12. Example Review Questions

A reviewer may ask:

- Which requirements are uncovered?
- Which high-priority requirements failed?
- Which tests cover authentication?
- Which source document supports this requirement?
- Which defects block release?
- Which tests are linked to obsolete requirements?
- What changed since the last baseline?

13. Common Anti-Patterns

13.1 Coverage by Title Match Only

Problem:

- a test title sounds related but does not verify the requirement.

Correction:

- inspect test steps and expected results.

13.2 Many-to-Many Without Explanation

Problem:

- a large group of tests and requirements are linked without clear rationale.

Correction:

- explain the relationship or split requirements/tests.

13.3 Ignoring Negative Tests

Problem:

- security and reliability requirements are only tested through happy paths.

Correction:

- add unauthorized, failure, timeout, malformed input, and boundary tests.

13.4 Stale Trace Links

Problem:

- requirement changes but test remains linked without review.

Correction:

- mark trace links as needing review after requirement baseline changes.

14. References

- NASA Systems Engineering Handbook guidance on verification matrices and traceability.
- NASA systems-engineering appendix examples of requirements verification matrices.
- ISO/IEC/IEEE 29148 requirement attributes and traceability concepts.